Sequence Models and Attention

Recurrent Networks, LSTM and GRU, Encoder-Decoder Models, and Attention as Search

Naeemullah Khan
naeemullah.khan@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

November 9, 2026

Table of Contents

1. Motivation: word order changes meaning, a fixed window cannot reach far enough, and every word needs its context
2. Recurrent networks: reading one word at a time, the update rule worked by hand, input-output shapes, padding and packing, and a classifier in PyTorch
3. Training recurrent networks: backpropagation through time, vanishing and exploding gradients, clipping
4. LSTM: a separate cell state, forget, input and output gates, and one step worked by hand
5. GRU: two gates and no separate cell, how it compares with the LSTM, and three classifiers measured on IMDb
6. Encoder-decoder models: read then write, teacher forcing, and the bottleneck
7. Attention: a fresh context vector at every step, learned alignments, and attention inside a review classifier
8. Queries, keys and values: attention as a soft lookup, self-attention and cross-attention, attention without recurrence
9. Summary and references

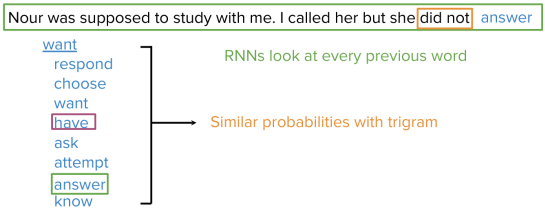
Word Order Changes Meaning

- Review A: “the plot was good, the acting was not bad”.
- Review B: “the plot was not good, the acting was bad”.
- A is positive and B is negative, yet they contain exactly the same words, the same number of times.
- A bag-of-words or TF-IDF vector gives them the **same vector**, so any classifier built on it must give them the same label.
- Bigram features help here (“not good” against “not bad”), but only for patterns that fit inside the window.

word	count in A	count in B
the	2	2
was	2	2
plot	1	1
acting	1	1
good	1	1
bad	1	1
not	1	1

Counts ignore order: both reviews map to the same row of a document-term matrix.

A Fixed Window Cannot Reach Far Enough

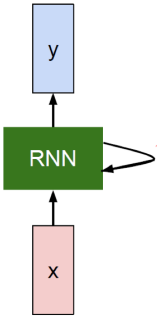


Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri and L. Kaiser).

- To predict "answer" the model needs "called her", five and six words back. A trigram model sees only "did not" and rates "have", "answer" and many others alike.
- A feed-forward network over the last k word embeddings (Bengio et al., 2003) also stops at k, and it learns separate weights for each position in the window.
- Widening the window multiplies the weights, and there is always a longer dependency: a negation at the start of a long review, the subject of a long sentence.

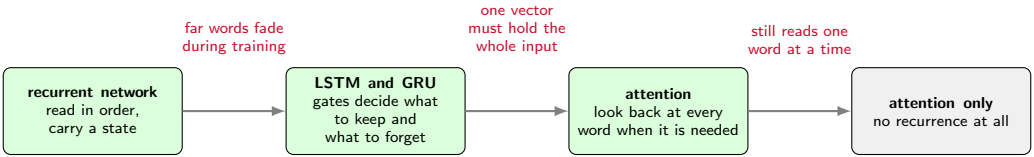
What a Sequence Model Needs

- **Any length:** reviews have 10 words or 2,000; sentences, 3 or 80.
- **Shared weights across positions:** what "not" does at position 3 should be what it does at position 300.
- **A memory:** a summary of everything read so far, carried forward one word at a time.
- **Context for every word:** *bank* in "the river bank" and in "the bank loan" should get different vectors. A table of word vectors gives both the same row; the vector must be computed from the words around it.
- A **recurrent neural network** (RNN) meets all four with one idea: feed the network's own previous state back in as an input at the next step. Its state after each word is a vector for that word in its context.



Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks.

Each Idea Fixes the Failure of the One Before



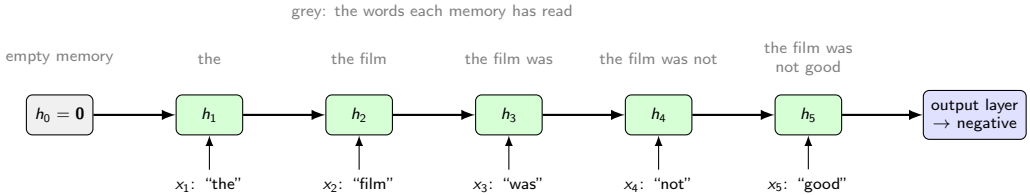
- The failures are concrete: a review whose verdict depends on a word 150 words back; a 50-word sentence to translate through one vector.
- The same task measures the steps: classifying the 25,000 test reviews of IMDb. A plain recurrent network gets 56.8% (a coin gets 50%), a gated one 84.5%.
- The last step, attention with no recurrence at all, is the design behind current language models.

Learning Outcomes

By the end of this lecture you should be able to:

- **Compute** a recurrent network's hidden states by hand, **count** its parameters, and **build** a recurrent review classifier in PyTorch.
- **Explain** why gradients vanish or explode over long sequences, and what gradient clipping fixes.
- **Trace** an LSTM gate step by hand, **explain** why gates let memory last longer, and **choose** between LSTM and GRU.
- **Describe** how an encoder-decoder model is trained with teacher forcing, and why one final state is a bottleneck.
- **Compute** attention weights and a context vector by hand, and **read** the attention weights of a review classifier with care.
- **Explain** attention as a soft lookup with queries, keys and values, and **distinguish** self-attention from cross-attention.
- **Prerequisite knowledge:** word embeddings; n-gram language models; bag-of-words and TF-IDF; feed-forward networks, softmax, cross-entropy and backpropagation; sigmoid and tanh; PyTorch.

An RNN Reads One Word at a Time and Updates a Memory



- Start with an empty memory h_0, a vector of zeros.
- At each word, one rule takes two inputs, the memory so far h_{t-1} and the embedding x_t of the new word, and returns the new memory h_t, called the **hidden state**.
- The rule and its weights are the same at every word: every green box in the picture is the same computation. That reuse is what "recurrent" means, and why one network reads 5 words or 2,000.
- h_t has read the first t words and nothing after them. After the last word, h_5 stands for the whole review, and an output layer reads the label from it.

The Update Rule, Term by Term

$$h_t = \tanh\Big(\underbrace{W_{hh}\, h_{t-1}}_{\text{what the memory carries forward}} + \underbrace{W_{xh}\, x_t}_{\text{what the new word adds}} + \underbrace{b_h}_{\text{bias}} \Big)$$

symbol	what it is	shape
x_t	embedding of word t	64
h_{t-1}, h_t	memory before, after word t	128
W_{xh}	writes the word in	128×64
W_{hh}	carries the old memory in	128×128
b_h	bias	128

Shapes for a review classifier with 64-dimensional embeddings and a 128-dimensional state.

- Both terms are added, so the new memory mixes what was known with what was just read.
- tanh squashes every entry into $(-1, 1)$: the memory stays bounded however many words are read, and the rule is not linear.
- An output is read only where a prediction is needed. A review label, after the last word: $\hat{y} = \sigma(w^\top h_T + b)$. The next word, at every step: $\hat{y}_t = \text{softmax}(W_{hy} h_t + b_y)$.
- This is the Elman network (Elman, 1990), often called a vanilla RNN.

A Two-Number Memory, Designed by Hand

- The task: tell "good" from "not good". Embeddings have two numbers, $x_{\text{not}} = (1, 0)$ and $x_{\text{good}} = (0, 1)$; biases are zero.
- The memory has two slots, each with a job we choose:
  - $h[1]$: how positive the text is so far; read it as the prediction $\hat{y}$.
  - $h[2]$: whether a "not" was read recently.
- Training would find the weights from data. Here we pick them so that each slot does its job.

$$W_{xh} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad W_{hh} = \begin{pmatrix} 1 & -2 \\ 0 & 0.5 \end{pmatrix}$$

Written out row by row, the update rule becomes:

$$h_t[1] = \tanh\left(h_{t-1}[1] - 2\, h_{t-1}[2] + x_t[2]\right)$$

keep the sentiment so far, subtract 2 if a "not" is remembered, add 1 for "good"

$$h_t[2] = \tanh\left(0.5\, h_{t-1}[2] + x_t[1]\right)$$

keep half of the remembered "not", add 1 for a new "not"

Running the Rule: The Same Word Meets a Different Memory

text	step	word	memory term $W_{hh}h_{t-1}$	word term $W_{xh}x_t$	$h_t = \tanh(\text{sum})$	$\hat{y} = h[1]$
"good"	1	good	$(0,\ 0)$	$(1,\ 0)$	$(0.762,\ 0)$	**+0.76**
"not good"	1	not	$(0,\ 0)$	$(0,\ 1)$	$(0,\ 0.762)$	
	2	good	$(-1.523,\ 0.381)$	$(1,\ 0)$	$(-0.480,\ 0.363)$	**−0.48**
"good not"	1	good	$(0,\ 0)$	$(1,\ 0)$	$(0.762,\ 0)$	
	2	not	$(0.762,\ 0)$	$(0,\ 1)$	$(0.642,\ 0.762)$	**+0.64**

- At step 2 of "not good", the word "good" adds the same $(1, 0)$ as it does alone. The memory term differs: the remembered "not" brings -1.523, and the verdict turns negative.
- "not good" and "good not" have the same words, so a bag of words gives them one vector. The RNN gives them opposite signs, because it reads them in order.
- A trained RNN with 128 slots works the same way, except that training sets the weights, and the slots have no jobs a person chose.

The Parameter Count Does Not Depend on Length

```
rnn = nn.RNN(64, 128)
for name, p in rnn.named_parameters():
    print(name, tuple(p.shape))
# weight_ih_l0 (128, 64)     W_xh
# weight_hh_l0 (128, 128)    W_hh
# bias_ih_l0 (128,)          two biases,
# bias_hh_l0 (128,)          added together
n = sum(p.numel() for p in rnn.parameters())
print(n)   # 24832
```

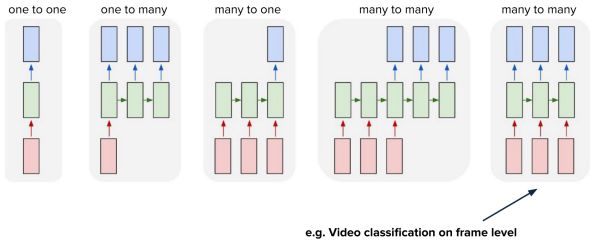
PyTorch keeps one bias per matrix; only their sum matters, so together they act as b_h.

- Every weight sits in the update rule:

$$\underbrace{128 \cdot 64}_{W_{xh}} + \underbrace{128^2}_{W_{hh}} + \underbrace{2 \cdot 128}_{\text{biases}} = 24{,}832$$

- The same 24,832 weights read a 10-word review or a 2,000-word one.
- A feed-forward layer over a 200-word window of the same embeddings, with 128 hidden units, needs $200 \cdot 64 \cdot 128 = 1{,}638{,}400$ weights, and still cannot see word 201.
- In a small classifier the embedding table dominates: $10{,}000 \times 64 = 640{,}000$ weights for a 10,000-word vocabulary.

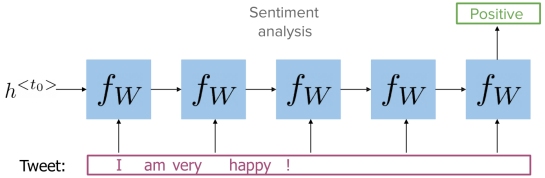
Four Ways to Wire Inputs and Outputs



Red: inputs; green: hidden states; blue: outputs. Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks; figure after A. Karpathy, "The Unreasonable Effectiveness of Recurrent Neural Networks" (2015).

shape	**NLP example**	**where the outputs are read**
one to many	image captioning	every step, from one image vector
many to one	sentiment of a review	the last state, or a pool of all states
many to many (delayed)	translation	a second network, after the input ends
many to many (aligned)	named entity tags, next word	every step, one output per input

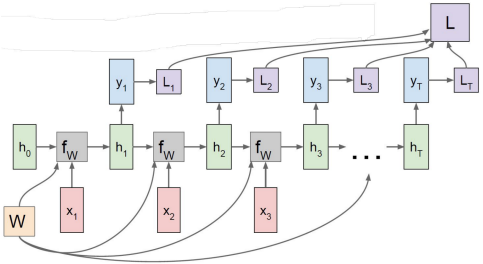
Many to One: Classifying a Review



Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri and L. Kaiser).

- Run the cell over every word; the state after the last word summarises the text.
- A linear layer and a sigmoid (two classes) or softmax (more) turn that state into a label; the loss is cross-entropy against the true label.
- Alternatives to the last state: the mean or the maximum of all states, which give early words a more direct path to the output.
- The gradient of the loss reaches the first word only through every step in between. That path is where training gets hard.

Many to Many: A Language Model Predicts Every Next Word



One loss L_t per step, summed into L. Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks.

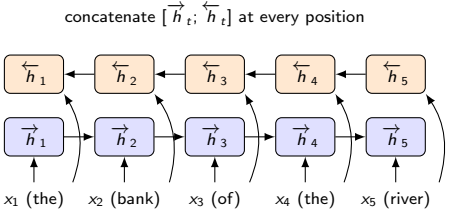
- Input: words $w_1 \dots w_T$. Target at step t: the next word w_{t+1}. The output layer has one score per vocabulary word.
- Loss, averaged over the sequence:

$$L = -\frac{1}{T} \sum_{t=1}^{T} \log \hat{y}_t \left[w_{t+1} \right]$$

- An n-gram model conditions on the last $n-1$ words. The RNN conditions on h_t, which in principle depends on every earlier word.
- Generation: sample a word from $\hat{y}_t$, feed it in as x_{t+1}, repeat.

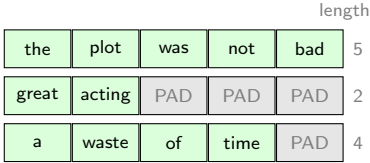
Bidirectional and Stacked RNNs

- A forward RNN's $\overrightarrow{h}_t$ has seen only $x_1 \dots x_t$. In "the bank of the river", the meaning of "bank" depends on words that come after it.
- A **bidirectional** RNN (Schuster and Paliwal, 1997) runs a second RNN from right to left and concatenates the two: $h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t]$.
- It works when the whole input is available first: tagging, classification, and reading a sentence before translating it. It cannot generate text, because the future words do not exist yet.
- **Stacking:** the states of layer 1 are the inputs of layer 2. The translation model of Sutskever et al. (2014) stacked four layers.



Blue: left-to-right states; orange: right-to-left states.

Reviews Have Different Lengths, but a Batch Is a Rectangle



one tensor: 3 reviews $\times$ 5 positions

Shorter reviews are filled up to the longest one with a reserved id, here 0, written PAD. It is not a word.

what could go wrong	the PyTorch fix
PAD gets an embedding that training changes, like a real word.	`nn.Embedding(..., padding_idx=0)`: that row is all zeros and never gets a gradient.
The RNN keeps stepping through the pads: a zero input still changes the state, through $W_{hh}h_{t-1}$ and the bias. The "last" state comes after the pads.	`pack_padded_sequence` with the true lengths: the RNN stops at each review's own last word.
With one label per word (tags, next words), the pad positions add to the loss.	Pad the labels with -100: `nn.CrossEntropyLoss` skips that label by default.

Padding and Packing in PyTorch, Checked

```
from torch.nn.utils.rnn import (
    pad_sequence, pack_padded_sequence)
torch.manual_seed(0)
emb = nn.Embedding(10_000, 64, padding_idx=0)
rnn = nn.RNN(64, 128, batch_first=True)

seqs = [torch.tensor([12, 7, 431, 9, 56]),
        torch.tensor([5, 88])]
lengths = torch.tensor([len(s) for s in seqs])
x = pad_sequence(seqs, batch_first=True)
# [[12, 7, 431, 9, 56], [5, 88, 0, 0, 0]]

_, h_pad = rnn(emb(x))     # runs over the pads
packed = pack_padded_sequence(
    emb(x), lengths, batch_first=True,
    enforce_sorted=False)
_, h_pack = rnn(packed)    # stops at true ends
_, h_true = rnn(emb(seqs[1])[None])  # alone

gap = lambda a, b: (a - b).abs().max()
gap(h_pad[0, 1], h_true[0, 0])     # 0.79
gap(h_pack[0, 1], h_true[0, 0])    # 2e-07
```

- pad_sequence pads with 0. With padding_idx=0, row 0 of the embedding is all zeros and its gradient is always zero.
- Run through its three pads, review 2 ends in a different state from review 2 read alone: entries differ by up to 0.79, and the cosine similarity is 0.27.
- Packed, it ends in the same state, up to rounding.
- Without packing, take each review's output at its own last position, `lengths - 1`, instead of the final state.

A Review Classifier in PyTorch

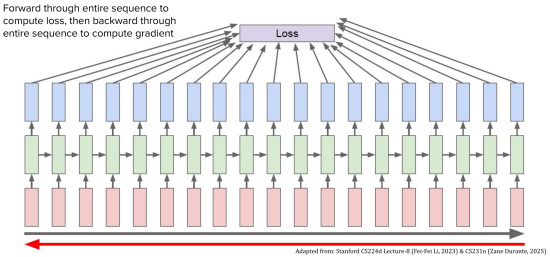
```
class RNNClassifier(nn.Module):
    def __init__(self, vocab, d_emb=64, d_h=128):
        super().__init__()
        self.emb = nn.Embedding(vocab, d_emb,
                                padding_idx=0)
        self.rnn = nn.RNN(d_emb, d_h,
                          batch_first=True)
        self.out = nn.Linear(d_h, 1)

    def forward(self, ids, lengths):
        packed = pack_padded_sequence(
            self.emb(ids), lengths,
            batch_first=True, enforce_sorted=False)
        _, h = self.rnn(packed)  # (1, batch, d_h)
        return self.out(h[-1]).squeeze(1)

model = RNNClassifier(10_000)
loss_fn = nn.BCEWithLogitsLoss()
opt = torch.optim.Adam(model.parameters(),
                       lr=1e-3)
loss = loss_fn(model(x, lengths), labels.float())
opt.zero_grad(); loss.backward(); opt.step()
```

- The whole model: an embedding table, one recurrent layer, and a linear layer on the state after the last word.
- `x` is a padded batch of ids, `lengths` the true lengths; `labels` holds 1 for a positive review and 0 for a negative one.
- `BCEWithLogitsLoss` applies the sigmoid inside the loss, which is more stable than a sigmoid followed by `BCELoss`.
- 664,961 weights, 640,000 of them in the embedding table. Training repeats the last two lines over every batch, for several passes over the data.

Training Runs Backwards Through Every Step



Source: Stanford CS231n lecture slides on recurrent networks, as credited in the figure.

- Unrolled over a review, an RNN is a deep network: one layer per word, every layer with the same weights.
- Training is ordinary backpropagation on that unrolled network, called **backpropagation through time** (BPTT). `loss.backward()` does it.
- For a review label, the loss comes after the last word. Its gradient reaches the first word only by passing back through every step in between.
- Every state is kept for the backward pass, so memory grows with the length. Long texts are cut, or backpropagated through only their last steps.

Each Step Multiplies the Gradient

- Each step back multiplies the gradient by the recurrent weights and by the slope of tanh, which is at most 1.
- Suppose every step multiplies it by the same factor a. After k steps it is multiplied by a^k; for a 200-word review, the first word's signal by a^{199}.
- **Below 1, it vanishes**, and silently: training runs, and the network learns short-range patterns only.
- **Above 1, it explodes**: sudden spikes in the loss, or not-a-number (NaN) values.

a	$k = 10$	$k = 50$	$k = 199$
0.5	9.8×10^{-4}	8.9×10^{-16}	$\approx 10^{-60}$
0.9	0.35	5.2×10^{-3}	7.8×10^{-10}
0.99	0.90	0.61	0.14
1.1	2.6	117	1.7×10^{8}

a^k computed directly. The slope of tanh is 1 at 0 and 0.42 at ± 1.

Gradient Clipping Fixes Explosions, Not Vanishing

- Compute the gradient g of all parameters together. If its norm exceeds a threshold τ, rescale it:

$$g \leftarrow \frac{\tau}{\|g\|}\, g \qquad \text{if } \|g\| > \tau$$

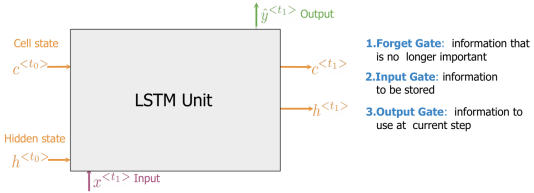
- The direction is kept and only the step length is capped, so one bad batch cannot throw the weights far away.
- In PyTorch, call `clip_grad_norm_` from `torch.nn.utils` on the model's parameters, between `backward()` and `step()`.

Pascanu, Mikolov and Bengio, "On the difficulty of training Recurrent Neural Networks," ICML 2013, arXiv:1211.5063v2, Algorithm 1.

Vanishing needs a different cell

- Clipping cannot help: a gradient that has shrunk to 10^{-10} carries no signal from the early words, and rescaling it leaves it carrying none.
- The cause is the update rule itself: every word rewrites the whole memory through W_{hh} and tanh, so every step back multiplies the gradient again.
- What would fix it: a memory that each word changes by **adding** to it, where a step back multiplies the gradient by a learned number between 0 and 1 that the network can hold close to 1.

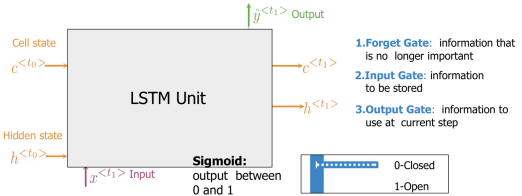
The LSTM Keeps a Second State: the Cell



Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 3 (Y. Bensouda Mourri and L. Kaiser).

- **The problem:** a plain RNN rewrites its whole memory at every word, so a fact from word 3, and its gradient, must survive 200 rewrites.
- **The fix:** the long short-term memory (LSTM) unit (Hochreiter and Schmidhuber, 1997) adds a second memory, the **cell state** c_t, that each word changes only by erasing part of it and adding to it. The **hidden state** h_t is what it shows to the next layer.
- Three **gates** decide, separately for every unit, what to erase from the cell, what to write into it, and what to read out of it. The forget gate was added by Gers, Schmidhuber and Cummins (2000).

A Gate Is a Sigmoid That Scales a Vector



Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 3 (Y. Bensouda Mourri and L. Kaiser).

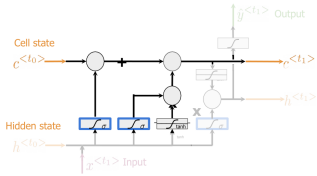
- A gate is a layer with a sigmoid output, one value in $(0, 1)$ per unit, computed from the previous hidden state and the current input.
- It multiplies another vector elementwise ($\odot$): 0 blocks that unit, 1 lets it through unchanged, 0.3 lets through 30%.
- Because the gate values depend on the input, the network decides at every word how much to keep, and the decision is learned by gradient descent like any other weight.

The LSTM Equations

With $z_t = [h_{t-1}; x_t]$, the previous hidden state and the input stacked into one vector:

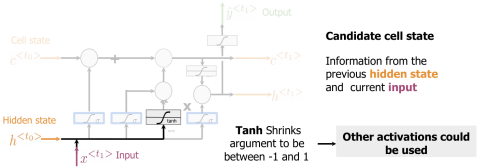
$$
\begin{aligned}
f_t &= \sigma\left(W_f z_t + b_f\right) && \text{forget gate} \\
i_t &= \sigma\left(W_i z_t + b_i\right) && \text{input gate} \\
\tilde{c}_t &= \tanh\left(W_c z_t + b_c\right) && \text{candidate} \\
o_t &= \sigma\left(W_o z_t + b_o\right) && \text{output gate} \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{new cell state} \\
h_t &= o_t \odot \tanh\left(c_t\right) && \text{new hidden state}
\end{aligned}
$$

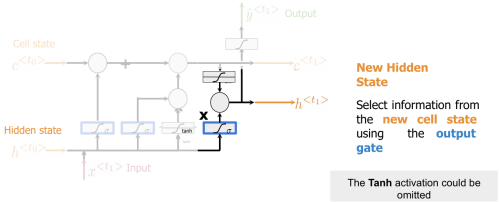
σ is the logistic sigmoid; every gate and state has d_h entries.



The new cell state: keep part of the old one (forget gate), add part of the candidate (input gate). Source: DeepLearning.AI NLP Specialization, Course 3, Week 3 (Y. Bensouda Mourri and L. Kaiser).

Writing to and Reading from the Cell





Candidate cell state $\tilde{c}_t$.

- The candidate is what the current word proposes to store, squashed to $(-1, 1)$ by tanh.
- The input gate decides how much of it enters: a word that carries no new information can leave the cell untouched.

New hidden state h_t.

- The output gate decides which parts of the memory are relevant now. A unit can hold a fact for 50 words while showing nothing of it.
- Predictions and the next layer read h_t; only the LSTM itself reads c_t.

Both figures: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 3 (Y. Bensouda Mourri and L. Kaiser).

One LSTM Step by Hand

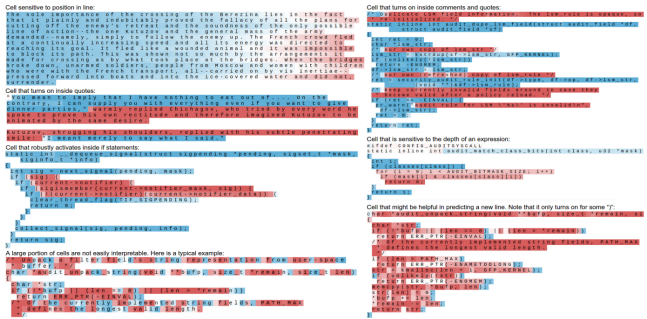
- One unit. Its cell holds $c_{t-1} = 0.9$, say a negation seen earlier in the review. The next word gives these gate pre-activations: forget 2.0, input -1.0, candidate 0.5, output 1.0.
- The forget gate keeps 88% of the memory, the input gate admits only 27% of the candidate: the memory survives almost intact.
- The cell changed by **addition**. Going back one step along the cell, the gradient is multiplied by $f_t = 0.881$ alone: no weight matrix, no tanh slope. A forget gate near 1 is a path the gradient can travel.
- **In practice:** a forget gate with bias 0 starts near $\sigma(0) = 0.5$ and halves the memory at every step. Starting its bias at 1 "closes the gap between the LSTM and the GRU" (Jozefowicz et al., 2015).

$$f_t = \sigma(2.0) = 0.881 \qquad i_t = \sigma(-1.0) = 0.269$$
$$\tilde{c}_t = \tanh(0.5) = 0.462 \qquad o_t = \sigma(1.0) = 0.731$$

$$c_t = f_t\, c_{t-1} + i_t\, \tilde{c}_t = 0.881 \cdot 0.9 + 0.269 \cdot 0.462$$
$$= 0.793 + 0.124 = 0.917$$
$$h_t = o_t \tanh(c_t) = 0.731 \cdot 0.724 = 0.530$$

With $f_t = 0.881$ at every step, a memory keeps $0.881^{50} = 0.0018$ of its size after 50 words; with $f_t = 0.99$, it keeps 0.61.

What LSTM Cells Learn to Track



Each panel colours text by one cell's value $\tanh(c)$, from -1 (red) to $+1$ (blue). Karpathy, Johnson and Fei-Fei, "Visualizing and Understanding Recurrent Networks," 2015, arXiv:1506.02078v2, Figure 2.

- Character-level LSTM language models trained on *War and Peace* and on Linux kernel source code.
- Some single cells keep track of "long-range dependencies such as line lengths, quotes and brackets": one switches on at an opening quote and stays on until the quote closes, across whole lines of text. Many other cells have no simple reading.

The GRU: Two Gates and No Separate Cell

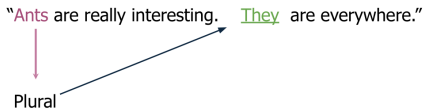
The LSTM carries two states and four weight blocks. The gated recurrent unit (GRU) of Cho et al. (2014) keeps the gates with less: one state h_t and two gates.

$$
\begin{aligned}
r_t &= \sigma\left(W_r x_t + U_r h_{t-1}\right) && \text{reset gate} \\
z_t &= \sigma\left(W_z x_t + U_z h_{t-1}\right) && \text{update gate} \\
\tilde{h}_t &= \tanh\left(W x_t + U\left(r_t \odot h_{t-1}\right)\right) && \text{candidate} \\
h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t && \text{new state}
\end{aligned}
$$

Biases are omitted. Some texts swap z_t and $1 - z_t$; PyTorch's `nn.GRU` uses the form above.

- The update gate mixes old and new: $z_t \approx 1$ copies the previous state forward unchanged, the role of an LSTM forget gate near 1.
- The reset gate decides how much of the old state the candidate may see. Near 0, "the hidden state is forced to ignore the previous hidden state and reset with the current input only" (Cho et al., 2014).
- Three weight blocks instead of the LSTM's four: 74,496 weights against 99,328 at $d_x = 64$, $d_h = 128$.

What the Update Gate Is For



Relevance and update gates to remember important prior information

Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri and L. Kaiser).

- To choose "They" over "It", the model must still know at "are everywhere" that "Ants" was plural.
- A unit that stores "plural subject" can set $z_t \approx 1$ on the words in between and so carry the fact across them unchanged.
- At the next subject it sets $z_t \approx 0$ and overwrites the fact with the new one.
- Each unit has its own gate value, so some units can hold slow facts while others track the last few words.

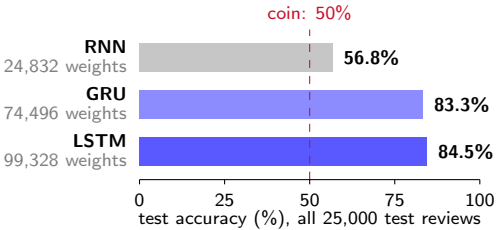
LSTM or GRU?

	RNN	GRU	LSTM
gates	0	2	3
states carried	h	h	h, c
weight blocks	1	3	4
weights at $d_h = 128$	24,832	74,496	99,328
additive memory path	no	yes	yes

Weights counted with PyTorch for this course, with 64-dimensional inputs.

- Chung et al. (2014) compared the units on music and speech modelling: both gated units clearly beat the tanh unit, but "we could not make concrete conclusion on which of the two gating units was better".
- Jozefowicz et al. (2015) searched over ten thousand variants; a forget-gate bias of 1 closed the gap between LSTM and GRU.
- In practice the GRU is smaller and a little faster, and neither wins everywhere, so try both on the validation set.

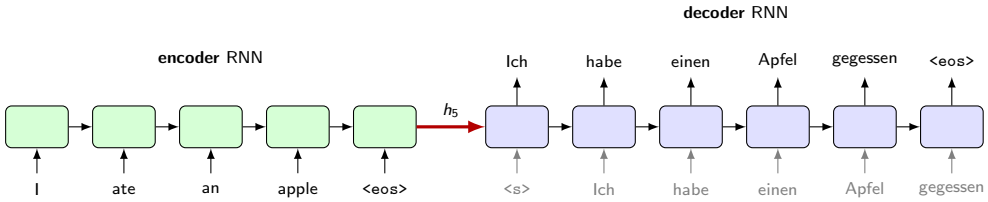
Three Recurrent Classifiers on IMDb, Measured



- One PyTorch classifier, trained three times with `nn.RNN`, `nn.GRU` and `nn.LSTM` (which also returns its cell: `_, (h, c) = ...`). Same data, sizes and training.
- The plain RNN barely beats a coin. Its label is read after up to 200 words, far more steps than a vanishing gradient survives.
- The gates add 27 points. The GRU comes within about a point of the LSTM with three quarters of its recurrent weights.
- Both still squeeze a whole review into the state after its last word.

IMDb reviews (Maas et al., 2011): 22,411 training reviews after removing repeats, 2,490 for validation, all 25,000 test reviews. Words split on spaces, 10,000-word vocabulary, reviews cut at 200 words; embeddings of 64, state of 128, 8 epochs of Adam at 10^{-3}, dropout 0.3 before the output layer of the GRU and LSTM. Computed for this course on CPU.

Encoder-Decoder: Read the Input, Then Write the Output



Redrawn after CMU 11-785 (Spring 2024), Lecture 18: Sequence to Sequence Models. Grey words: the decoder's input at each step, its own previous output.

- A classifier writes one label. Translation must write a new sentence, of its own length: one network reads, a second one writes until it emits the end-of-sequence token <eos>.
- The **encoder** reads the source sentence; its final state h_5 is the only thing passed on. The **decoder** is a language model of the target language whose first state is h_5.
- Sutskever, Vinyals and Le (2014) and Cho et al. (2014) introduced this design for translation; it also fits summarisation, dialogue and speech recognition.

Training with Teacher Forcing

- The model is trained to maximise the probability of the reference translation $y_1 \dots y_m$ given the source x:

$$L = -\sum_{t=1}^{m} \log p\left(y_t \mid y_{<t}, x\right)$$

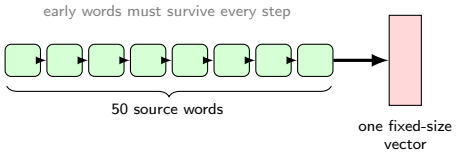
- **Teacher forcing** (Williams and Zipser, 1989): at training time the decoder's input at step t is the **reference** word y_{t-1}, whatever the model predicted at step $t-1$.
- Every step then has a correct prefix to learn from, and all m losses come from one pass over the reference.
- At test time no reference exists and the decoder must read its own outputs. Errors it never saw in training can compound: this mismatch is called **exposure bias**.

step	decoder input	target
1	<s>	Ich
2	Ich	habe
3	habe	einen
4	einen	Apfel
5	Apfel	gegessen
6	gegessen	<eos>

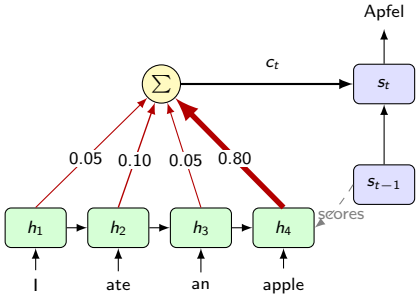
Under teacher forcing the input column is the target column shifted down by one step, starting with a start token <s>.

The Bottleneck: One Vector for the Whole Sentence

- Everything the decoder knows about the source must pass through the encoder's final state, a vector of fixed size, whether the sentence has 5 words or 50.
- The first words of the source must survive every encoder step to reach it, and the first words of the output depend on them.
- Sutskever et al. (2014) found that **reversing the source sentence** "improved the LSTM's performance markedly", because it puts the first source words next to the first target words.
- Their model was large: four LSTM layers of 1,000 cells, so 8,000 numbers represent a sentence, and 384M parameters.
- Bahdanau, Cho and Bengio (2014) called the fixed-length vector "a bottleneck in improving the performance of this basic encoder–decoder architecture".



Attention Removes the Bottleneck: Look Back at Every Word



Redrawn after Bahdanau et al. (2014), Figure 1. The weights shown are illustrative.

- **The problem:** the decoder sees the source only through h_n, one vector of fixed size.
- **The fix, attention** (Bahdanau, Cho and Bengio, 2014): keep **all** encoder states $h_1 \dots h_n$, one per source word.
- Before producing each output word, the decoder scores every h_i against its own state and takes a weighted average of the h_i: a **context vector** c_t made for that step.
- Writing "Apfel", most of the weight goes to "apple"; writing "gegessen", to "ate". Every source word is one step away from every output word.

Attention in Three Steps

without attention: every decoder step reads the same summary of the source, $c = h_n$.
with attention: decoder step t builds its own, $c_t = \sum_i \alpha_{t,i}\, h_i$, with weights made for that step.

At decoder step t, with previous decoder state s_{t-1} and encoder states $h_1 \dots h_n$:

$$e_{t,i} = \text{score}\left(s_{t-1}, h_i\right) \qquad \text{1. score each source position}$$

$$\alpha_{t,i} = \frac{\exp\left(e_{t,i}\right)}{\sum_{j=1}^{n} \exp\left(e_{t,j}\right)} \qquad \text{2. softmax into weights}$$

$$c_t = \sum_{i=1}^{n} \alpha_{t,i}\, h_i \qquad \text{3. weighted sum}$$

The decoder then computes s_t from s_{t-1}, the previous output word y_{t-1} and c_t, and predicts y_t.

- The weights are positive and sum to 1: a probability distribution over source positions, recomputed at every output step.
- Everything is differentiable, so the scoring function is trained jointly with the encoder and decoder from translation pairs alone. Nobody labels which word aligns with which.
- Bahdanau et al. used a bidirectional encoder, so each h_i summarises the words on both sides of x_i.
- Cost: $n \times m$ scores for a source of n words and an output of m.

Attention by Hand

- Three encoder states in two dimensions:

$$h_{\text{I}} = (1, 0), \quad h_{\text{ate}} = (0, 1), \quad h_{\text{apple}} = (1, 1)$$

- Decoder state about to write "Apfel": $s = (2, 1)$.
- Score with the dot product $e_i = s^\top h_i$.
- Two thirds of the weight lands on "apple", and the context vector is pulled towards h_{apple}.

1. Scores $e = (2,\ 1,\ 3)$

2. Softmax $e^2 = 7.389$, $e^1 = 2.718$, $e^3 = 20.086$, sum 30.193

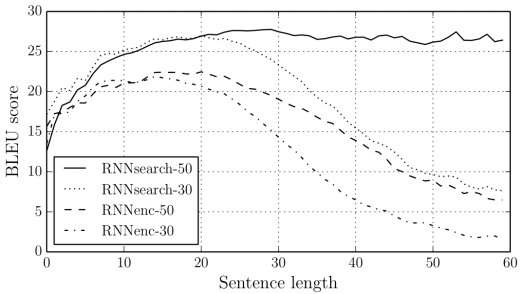
$$\alpha = (0.245,\ 0.090,\ 0.665)$$

3. Context

$$\begin{aligned} c &= 0.245\,(1, 0) + 0.090\,(0, 1) + 0.665\,(1, 1) \\ &= (0.910,\ 0.755) \end{aligned}$$

Doubling s to $(4, 2)$ gives scores $(4, 2, 6)$ and weights $(0.117, 0.016, 0.867)$: larger scores make the softmax sharper.

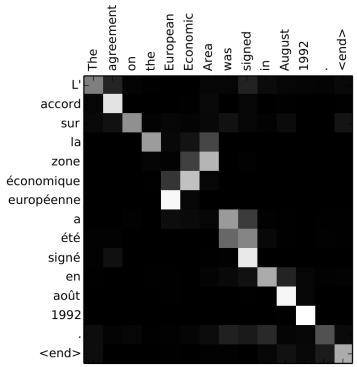
Attention Keeps Long Sentences Intact



Bahdanau, Cho and Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," ICLR 2015, arXiv:1409.0473v7, Figure 2.

- English to French, on the 3,003-sentence test set of the 2014 Workshop on Machine Translation. RNNenc: encoder-decoder with one final state; RNNsearch: with attention. The suffix is the longest training sentence (30 or 50 words).
- Without attention, BLEU falls steadily once sentences pass about 20 words. With attention, RNNsearch-50 stays flat up to 60.
- On the whole test set: 26.75 BLEU against 17.82 for the same size of model without attention.

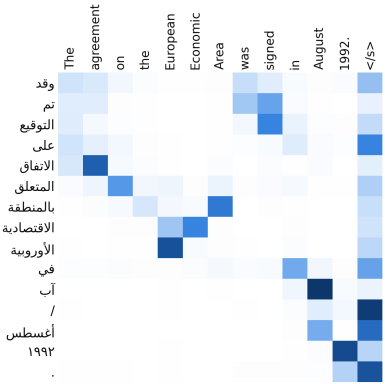
What Attention Learns: Soft Alignment



Bahdanau et al., arXiv:1409.0473v7, Figure 3(a).

- Each row is one French output word, each column one English source word, and each pixel the weight $\alpha_{t,i}$ (white = 1, black = 0).
- Mostly diagonal, because French and English often share word order.
- "European Economic Area" becomes "zone économique européenne": the weights run backwards across those three words, following the reversed order.
- "a été signé" spreads over "was signed": one source word can feed several outputs, and the alignment is soft.
- The model was never shown an alignment; it emerged from training on sentence pairs.

The Same Sentence, from English to Arabic



Computed for this course with `Helsinki-NLP/opus-mt-en-ar`; decoder attention over the source in the fifth of its six layers, averaged over that layer's 8 attention maps.

- A current English-to-Arabic translation model (77M parameters) translated the sentence of the previous figure; rows are the Arabic words it wrote.
- Arabic puts adjectives after the noun, like French: "المنطقة الاقتصادية الأوروبية" looks at "Area", "Economic", "European" in turn.
- Words with no single English source, such as وقد and the slash in آب, put their weight on the end token </s>.
- This model has no recurrence at all, only attention layers, yet its decoder consults the source in the same way.

Attention Inside a Review Classifier

A positive review (predicted positive)

amazing movie some script writing could have better some cliched language joyce s dead alluded throughout movie beautiful scenery great acting **poetic** 0.11 **highly** 0.22 **recommend** 0.11

A negative review (predicted negative)

far worst non english horror movie ve ever seen acting **wooden** 0.20 dialogues simply stupid story totally braindead s not even **scary** 0.11 2 out 10 from

Two IMDb test reviews, words as the model reads them: lower-cased, without punctuation or 42 stop words ("I've" becomes "ve"). Darker red: more attention weight; weights of 0.10 and above are printed. Computed for this course.

- The recurrent classifiers share the bottleneck: the output layer sees only the last state. With attention, the last state scores every word's state, and the weighted sum goes to the output layer.
- The weights find the verdict: "highly recommend", "wooden", "not even scary".
- Yet "amazing" and "worst" get 0.02 and 0.03. Each state already carries the words read before it, so weight on "highly" is weight on everything up to "highly".
- Measured: 83.8% on all 25,000 test reviews, about what the GRU alone reached. What attention added here is a view of the positions used.

GRU with attention: embeddings and state of 128, reviews cut at 200 words, 10 epochs of Adam at 3×10^{-4}, 9.4M weights; 22,500 IMDb training reviews (Maas et al., 2011), 2,500 for validation. Trained for this course on CPU.

Attention Weights Are Not an Explanation

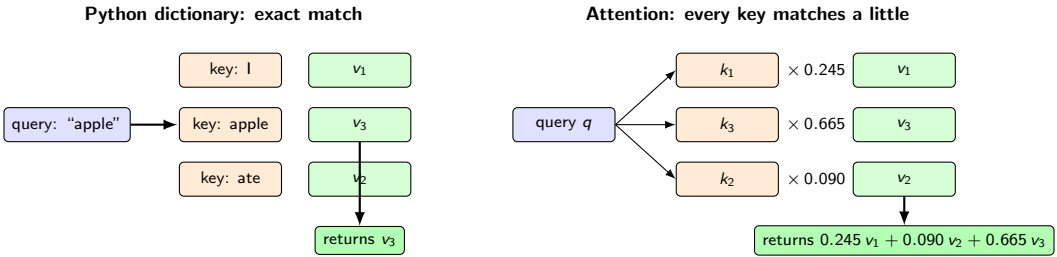
- It is tempting to read a heatmap of attention weights as "the words the model used".
- Jain and Wallace (2019) tested this with bidirectional LSTM models with attention on text classification, question answering and natural language inference: weights were "frequently uncorrelated with gradient-based measures of feature importance", and very different weights could give the same prediction.
- Wiegreffe and Pinter (2019) replied that the claim "depends on one's definition of explanation" and that testing it must consider the whole model, and proposed four alternative tests.

What a heatmap does and does not show

- It shows which positions were mixed into one context vector at one layer.
- It does not show what was inside those vectors: after a bidirectional encoder, h_i already contains information about the words around x_i.
- Treat attention maps as a useful view of what was combined, and check claims about why a model decided something with other methods.

Jain and Wallace, "Attention is not Explanation," NAACL 2019, arXiv:1902.10186v3; Wiegreffe and Pinter, "Attention is not not Explanation," EMNLP 2019, arXiv:1908.04626v2.

Attention Is a Soft Dictionary Lookup



- Attention fixed the bottleneck, but the encoder still reads one word at a time. Letting words look at each other directly needs attention in general terms: a lookup. A dictionary compares a query with each key and returns the value of the one key that matches.
- Attention compares the query with every key, turns the scores into weights with a softmax, and returns the weighted average of all values. It never fails, and it is differentiable, so the model can learn what to look for.
- Weights shown: the decoder state $s = (2, 1)$ as the query, the encoder states of "I", "ate", "apple" as both keys and values.

Queries, Keys and Values Have Separate Roles

- Using one vector both to be matched and to be returned forces it to serve two purposes. Three learned projections separate them:

$$q = W_Q\, s, \qquad k_i = W_K\, h_i, \qquad v_i = W_V\, h_i$$

- **Query** q: what the current position is looking for. **Key** k_i: what position i offers to be found by. **Value** v_i: what position i hands over when it is chosen.
- With the scaled dot product as the score:

$$\text{Attention}(q, K, V) = \sum_i \text{softmax}_i\left(\frac{q^\top k_i}{\sqrt{d_k}}\right) v_i$$

where d_k is the length of the query and key vectors. Dot products of long vectors are large numbers, which would push the softmax onto one position; dividing by $\sqrt{d_k}$ keeps them in range.

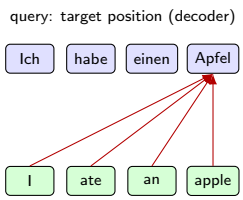
Search engine analogy

query	the text typed into the box
keys	titles and tags of the pages
values	the pages themselves
weights	how well each title matches

A page is found by its title but what you read is its content. In attention, both are learned, and the result is a blend of all pages instead of a ranked list.

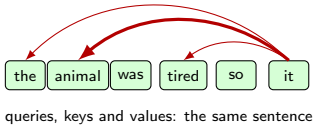
Self-Attention and Cross-Attention

Cross-attention



- Queries from one sequence, keys and values from another. This is the encoder-decoder attention of translation.

Self-attention



- Every position queries every position of its own sequence, and is rewritten as a mix of them: "it" can take in what "animal" means.

Arrow thickness sketches attention weights; the self-attention weights are illustrative.

Attention Without Recurrence

- With attention, the decoder no longer depends on the encoder's last state. The encoder states themselves are still computed one after another, so a long text needs a long chain of sequential steps.
- Self-attention can build those states instead: every word is re-expressed from all words, every position at once.
- Any two words are then one step apart, whatever their distance, so the gradient between them passes through one layer instead of hundreds of steps.
- Vaswani et al. (2017) built a translation model this way, with no recurrence at all: the transformer.

What self-attention alone does not give

- **Order:** a weighted average does not care where a word sits. Shuffle the words and each one gets the same output, so word positions must be supplied separately.
- **Cheap long inputs:** n positions give n^2 scores; doubling the text quadruples the score matrix.
- **A causal direction:** for generation, a position must not attend to the words after it.

Summary

- **Order matters, windows are too short, and every word needs its context.** A recurrent network reads any length with one set of weights and carries a state forward: $h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$.
- **One cell, several shapes.** Read the last state for a label, every state for tags or next words, or hand the last state to a second network to write a new sequence. In PyTorch a review classifier is an embedding table, `nn.RNN` and one linear layer; `padding_idx` keeps the pad vector at zero, and packing makes the last state belong to the last real word.
- **Every step multiplies the gradient.** Below 1 it vanishes silently, above 1 it explodes. Clipping the gradient norm fixes explosions; it cannot restore a vanished signal.
- **Gates let memory last.** The LSTM updates its cell by addition, $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$; with the forget gate near 1, memory and gradient pass almost unchanged. The GRU does the same with an update gate and fewer weights.
- **Measured on IMDb:** the plain RNN gets 56.8%, barely above a coin; the GRU 83.3% and the LSTM 84.5%.
- **Encoder-decoder models** read the source, then write the target, trained with teacher forcing. One final state is a bottleneck for long sentences.
- **Attention builds a fresh context at every step:** score, softmax, weighted sum. It kept translation quality flat up to 60-word sentences, learns soft alignments, and lets a classifier look back at every word. Its weights show what was mixed, not why the model decided.
- **Attention is a soft lookup:** queries are matched against keys and return a blend of values, $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$. Self-attention takes all three from one sequence, which removes the need for recurrence but leaves word order to be supplied.

References

1. Bengio, Ducharme, Vincent and Jauvin, "A Neural Probabilistic Language Model," *JMLR* 3:1137–1155, 2003. jmlr.org/papers/v3/bengio03a
2. DeepLearning.AI, Natural Language Processing Specialization, Course 3: *Natural Language Processing with Sequence Models*, Weeks 2–3 (source of the figures so credited). deeplearning.ai
3. Stanford CS231n, *Deep Learning for Computer Vision*, lecture on recurrent neural networks (source of the figures so credited). cs231n.stanford.edu
4. Elman, "Finding Structure in Time," *Cognitive Science* 14(2):179–211, 1990. doi:10.1207/s15516709cog1402_1
5. Karpathy, "The Unreasonable Effectiveness of Recurrent Neural Networks," 2015. karpathy.github.io
6. Schuster and Paliwal, "Bidirectional Recurrent Neural Networks," *IEEE Transactions on Signal Processing* 45(11):2673–2681, 1997. doi:10.1109/78.650093
7. Sutskever, Vinyals and Le, "Sequence to Sequence Learning with Neural Networks," NeurIPS 2014. arXiv:1409.3215v3
8. Pascanu, Mikolov and Bengio, "On the difficulty of training Recurrent Neural Networks," ICML 2013. arXiv:1211.5063v2
9. Maas et al., "Learning Word Vectors for Sentiment Analysis" (the IMDb review dataset), ACL 2011. aclanthology.org/P11-1015

References (cont.)

10. Hochreiter and Schmidhuber, "Long Short-Term Memory," *Neural Computation* 9(8):1735–1780, 1997. doi:10.1162/neco.1997.9.8.1735

11. Gers, Schmidhuber and Cummins, "Learning to Forget: Continual Prediction with LSTM," *Neural Computation* 12(10):2451–2471, 2000. doi:10.1162/089976600300015015

12. Jozefowicz, Zaremba and Sutskever, "An Empirical Exploration of Recurrent Network Architectures," ICML 2015, PMLR 37:2342–2350. proceedings.mlr.press/v37/jozefowicz15

13. Karpathy, Johnson and Fei-Fei, "Visualizing and Understanding Recurrent Networks," 2015. arXiv:1506.02078v2 (source of Figure 2)

14. Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," EMNLP 2014. arXiv:1406.1078v3

15. Chung, Gulcehre, Cho and Bengio, "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling," NeurIPS 2014 Deep Learning and Representation Learning Workshop. arXiv:1412.3555v1

16. CMU 11-785, *Introduction to Deep Learning*, Spring 2024, Lecture 18: Sequence to Sequence Models (source of the redrawn encoder-decoder figure). deeplearning.cs.cmu.edu

17. Williams and Zipser, "A Learning Algorithm for Continually Running Fully Recurrent Neural Networks," *Neural Computation* 1(2):270–280, 1989. doi:10.1162/neco.1989.1.2.270

References (cont.)

18. Bahdanau, Cho and Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," ICLR 2015. arXiv:1409.0473v7 (source of Figures 1, 2 and 3)

19. Tiedemann and Thottingal, "OPUS-MT – Building open translation services for the World," EAMT 2020 (the `opus-mt-en-ar` model). aclanthology.org/2020.eamt-1.61

20. Jain and Wallace, "Attention is not Explanation," NAACL 2019. arXiv:1902.10186v3

21. Wiegreffe and Pinter, "Attention is not not Explanation," EMNLP 2019. arXiv:1908.04626v2

22. Vaswani et al., "Attention Is All You Need," NeurIPS 2017. arXiv:1706.03762v7